

Measuring a Synchronous API Contract

An empirical investigation of request lifetime, work duration, and client waiting time

Duration	Work mode	Code
75 minutes	Pairs	Provided

Central question How does report-generation time affect the externally observable behavior of a synchronous report API?

Learning outcomes

- Measure API response time under controlled conditions.
- Distinguish a prediction, an observation, and a causal explanation.
- Identify the relationship between work duration and response time.
- Explain how the synchronous contract couples request lifetime and work lifetime.
- Derive a system requirement from measured evidence.

The current contract

A contract is an externally observable promise between components. It states what a client may expect from a server; it does not describe the server's internal implementation.

Current promise If the request succeeds, POST /v1/reports returns 200 OK with the completed report in the response body.

Because the completed report must be placed in the response, the handler must finish report generation before it can fulfill the contract. The client therefore waits while the work is performed.

Approximate relationship:

$$\text{response time} \approx \text{HTTP overhead} + \text{report-generation time}$$

Experimental design

Research question

How does report-generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis — complete before testing

If report-generation time increases, then response time will increase by a similar amount because the server can't send its response until the report is fully generated.

Variables

Role	Variable
Independent	Artificial report-generation delay: 0 s, 3 s, 7 s, 12 s
Dependent	Response time, HTTP status, response bytes, complete report received, server completion log
Controlled	Endpoint, request body, consumer, reporting period, database contents, client command and server timeout

Before you measure: make predictions

Record predictions before running the server under each condition. Predictions are not graded for being correct; they are evidence of your reasoning before observation.

Delay	Predicted response time	Predicted status	Complete body?
0 s	Instant,	200	Yes
3 s	3.05	200	Yes
7 s	7.05s	200	Yes
12 s	Timeout/ Failure	000	No

At what point do you predict the current design will fail or become unreliable? Why?

I predicted the design would fail or become unreliable somewhere in the 10-15 second range, since that is a normal connection timeout range.

Procedure

Part A — Start one experimental condition

Your instructor will provide the database connection value. Start the API with one artificial-delay condition:

```
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=0s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=3s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=7s
go run ./cmd/api -db-dsn="$GATEKEEPER_DB_DSN" -report-delay=12s
```

Controlled experiment Change only the report-delay value. Restart the server between conditions. Keep the request, data and measurement command unchanged.

Part B — Send and measure the request

In a second terminal, create request.json once:

```
{
  "consumer_id": "0198f000-0000-7000-8000-000000000001",
  "from": "2026-01-01T00:00:00Z",
  "to": "2027-01-01T00:00:00Z"
}
```

Then run the same measurement command for every condition:

```
curl --silent --show-error \
  --output response.json \
  --write-out 'status=%{http_code}\ntime=%{time_total}s\nbytes=%{size_download}\n' \
  --request POST \
  --header 'Content-Type: application/json' \
  --data @request.json \
  http://localhost:4000/v1/reports
```

After every request, inspect the body:

```
cat response.json
```

Part C — Repeat consistently

1. Run the request three times for the current delay.
2. Record every result; do not discard unexpected results.
3. Inspect response.json and the server log after each request.
4. Stop and restart the server with the next delay.
5. Do not run different delay conditions simultaneously.

Raw measurements

Record the client-visible result and the server-visible result. A blank body, status 000, timeout or connection error is still an observation and must be recorded.

Condition: 0 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	0.022956s	319	Yes	Yes
2	200	0.002762s	319	Yes	Yes
3	200	0.002578s	319	Yes	Yes

Condition: 3 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	3.023230	319	Yes	Yes
2	200	3.005297	319	Yes	Yes
3	200	3.005254	319	Yes	Yes

Condition: 7 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	7.027112	319	Yes	Yes
2	200	7.009078	319	Yes	Yes
3	200	7.008094	319	Yes	Yes

Condition: 12 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	000	12.018130	0	No	Yes
2	000	12.005053	0	No	Yes
3	000	12.005243	0	No	Yes

Summarize the evidence

For each condition, calculate the approximate mean response time:

$$\text{mean response time} = (T_1 + T_2 + T_3) \div 3$$

Delay	Mean response time	Successful responses	Complete reports
0 s	0.009432s	3 / 3	3 / 3
3 s	3.011260s	3 / 3	3 / 3
7 s	7.014761s	3 / 3	3 / 3
12 s	12.009475s	0 / 3	0 / 3

Observation

Write only what you measured. Do not explain the mechanism yet.

When the report delay increased from **0 seconds** to **7 seconds**, the response time scaled accordingly, and every request returned a 200 OK status with a full report. However, at the **12-second** mark, this behavior broke down entirely. The mean response time hovered around **12.009 seconds**, but all three requests failed. They returned an HTTP status of 000 (Empty reply from server) with 0 bytes downloaded, meaning the client received no report data.

Explanation

Now explain the mechanism that produced the observations.

Why did response time change as the artificial delay increased?

Response time increased by almost exactly the same amount as the artificial delay because the delay is inserted directly inside the request-handling path, before the response is written. The handler calls `time.Sleep(reportDelay)` and only generates and returns the report afterward.

What must the handler complete before it can send 200 OK with the report?

The handler must finish the entire report-generation process, including the artificial delay and the database query that builds the report data before it can write any part of the response. The 200 status and the report body are sent together, only after all of that work is done.

While the report is being generated, is the original request complete or outstanding?

The request is still open and unresolved for the entire duration of report generation. The client's connection stays alive waiting, and the request is not considered complete until the server finishes its work and sends the response back.

Would asynchronous execution make the report faster, or would it change when the client receives acknowledgement?

Asynchronous execution would not make the report generation itself any faster, the actual work still takes the same amount of time. What it would change is when the client is told the request was received: instead of waiting the full duration for one combined response, the client could get an immediate response that the job was accepted, and check on completion separately later.

Do not confuse cause with threshold A 10-second write timeout may expose the problem, but 10 seconds is not a universal boundary. Increasing the timeout moves the boundary; it does not remove the coupling.

From evidence to requirements

A requirement describes a property the system must provide. Do not write implementation mechanisms such as 202, worker, queue or polling in this section.

A valid report request should be acknowledged within a short and predictable amount of time, independent of how long the underlying report generation takes.

Report generation should not depend on the client staying connected and waiting for the entire duration of the work.

The client should be able to determine whether their request was received and is being processed, without needing to keep an open connection.

The system should preserve the outcome of the work (success or failure) so it can be retrieved after, rather than only being deliverable at the exact moment when generation finishes.

Report-generation duration should not determine how long the client is forced to wait before receiving any response at all.

The contract decision

The current contract promises completion in the original response. Based on your evidence and derived requirements, choose one option and justify it.

☐ Keep the current contract: return 200 OK with the completed report.

☒ Change the contract: acknowledge accepted work before the report is complete.

Justification:

The current contract must be changed because it strictly couples the request lifetime to the work duration. As our data shows, the response time scaled almost exactly 1:1 with the artificial delay for the 0s, 3s, and 7s conditions. However, at the 12-second delay, the system failed entirely, returning an HTTP status of 000 and zero bytes because the work duration exceeded the server's timeout configuration. This proves that a synchronous design is unreliable for long-running tasks; it forces the client to hold an open connection, which will eventually be killed by network or server timeouts before the work is finished. Separating acknowledgement from completion would let the client know its request was received immediately, without report-generation time dictating how long it waits for any response, directly satisfying the derived requirements.

In-class exit ticket

Submit one response per pair before leaving.

Element	Your response
Claim	The synchronous API is / is not suitable for potentially long-running reports because <u>Response time scales directly with report-generation time, forcing the client to wait however long the work takes with no way to receive earlier acknowledgement.</u>
Evidence	Provide at least two measured values. At a 0s delay, response time was 0.009s and succeeded. At a 12s delay, the request failed entirely with status 000 and 0 bytes downloaded because the wait time exceeded the server timeout.
Reasoning	Explain how the evidence follows from the current contract. <u>Because the contract requires the completed report to be placed in the same response that acknowledges the request, the handler cannot reply until report generation fully finishes.</u>
Requirement	State one requirement the current contract cannot reliably satisfy. <u>The current contract cannot reliably guarantee that a client is acknowledged within a short amount of time, since acknowledgement is tied to the full duration of report generation.</u>

Complete outside class

Submit a short laboratory report containing the following sections:

- Research question and hypothesis
- Independent, dependent and controlled variables
- Raw measurements and calculated mean response times
- Observations stated without explanation
- Causal explanation of the observed behavior

At least two limitations of the experiment
 Requirements derived from the evidence
 A proposed revised contract

Required reasoning chain

Layer	Question to answer
Evidence	What did we measure?
Problem	What coupling did the evidence expose?
Requirement	What property does the system now need?
Contract	What should the API promise the client?
Implementation	What mechanisms might provide that contract?

Important distinctions

Incorrect conclusion	Stronger conclusion
Async makes report generation faster.	Async can separate acknowledgement time from completion time; the work may take just as long.
The design fails after exactly 10 seconds.	The observed threshold depends on configuration; the underlying issue is lifetime coupling.
The timeout is the whole problem.	A larger timeout preserves the synchronous contract and merely permits longer waiting.
202 means the report succeeded.	202 means the server accepted the work; completion or failure must be observed later.

Next laboratory You will redesign the contract using Job + Worker + 202 Accepted + GET /jobs/{id}, then repeat the measurements and compare acknowledgement time with completion time.

CMPS4191 — Laboratory 1

Measuring a Synchronous API Contract

Kelvin Gordon - 201817973

Teryn Guzman - 2022156098

Research Question and Hypothesis

Research question: How does report-generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis: If report-generation time increases, then response time will increase by roughly the same amount, because the server cannot send its response until the report is fully generated — the client is stuck waiting for that work to finish before it can receive anything.

Variables

Independent Variable: Artificial report-generation delay: 0s, 3s, 7s, 12s.

Dependent Variable: Response time, HTTP status, response bytes, complete report received, server completion log.

Controlled Variable: Endpoint (POST /reports), request body, consumer (Belize City Council), reporting period, database contents, client command (curl), server timeout.

Raw measurements

Record the client-visible result and the server-visible result. A blank body, status 000, timeout or connection error is still an observation and must be recorded.

Condition: 0 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	0.022956s	319	Yes	Yes
2	200	0.002762s	319	Yes	Yes
3	200	0.002578s	318	Yes	Yes

Condition: 3 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	3.023230	319	Yes	Yes
2	200	3.005297	319	Yes	Yes
3	200	3.005254	319	Yes	Yes

Condition: 7 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	7.027112	319	Yes	Yes
2	200	7.009078	319	Yes	Yes
3	200	7.008094	319	Yes	Yes

Condition: 12 seconds

Run	HTTP status	Time (s)	Bytes	Complete report?	Server logged finish?
1	200	12.030932	319	Yes	Yes
2	200	12.014267	319	Yes	Yes
3	200	12.013322	318	Yes	Yes

Calculated mean response times

Delay	Mean response time	Successful responses	Complete reports
0 s	0.009432s	3 / 3	3 / 3
3 s	3.011260s	3 / 3	3 / 3
7 s	7.014761s	3 / 3	3 / 3
12 s	12.019507s	3 / 3	3 / 3

Observations stated without explanation

When report delay increased from **0 seconds** to **12 seconds**, mean response time changed from approximately **0.009432s** to **12.019507s**. This increase was consistent across all conditions as well: At 3 seconds delay, mean response time was approximately **3.011260s**; at 7 seconds delay, it was approximately **7.014761s**. In every condition, all three requests returned HTTP status 200 with a complete report body; no failures or timeouts were observed during any of the tests.

Causal explanation of the observed behavior

Response time increased by almost exactly the same amount as the artificial delay because the delay is inserted directly inside the request-handling code path, before the response is written. The handler sleeps for the configured delay and only proceeds to generate and return the report afterward, so every second added to the delay is a second the client is forced to wait, with no other part of the system able to absorb or hide that time.

The handler must finish the entire report-generation process, including the artificial delay and the database query that builds the report data before it can write any part of the response. The 200 status and the report body are sent together, only after all of that work is done.

While the report is being generated, the request is still open and unresolved for the entire duration. The client's connection stays alive waiting, and the request is not considered complete until the server finishes its work and sends the response back.

Asynchronous execution would not make the report generation itself any faster, the actual work still takes the same amount of time. What it would change is when the client is told the request was received: instead of waiting the full duration for one combined response, the client could get an immediate response that the job was accepted, and check on completion separately later.

At least two limitations of the experiment

Small sample size per condition. Only three requests were measured per delay, they ran sequentially on a single machine. This is not enough to characterize a difference under real-world conditions such as network jitter, concurrent load, or database contention, and the results may not generalize beyond this specific low-traffic, single-client setup.

The exact failure threshold observed (between 7 and 12 seconds) is an artifact of the specific `WriteTimeout` configuration of our server, rather than a universal constant. Changing the server or network timeout settings would shift this boundary, though it would not fix the underlying architectural coupling.

Requirements derived from the evidence

A valid report request should be acknowledged within a short and predictable amount of time, independent of how long the underlying report generation takes.

Report generation should not depend on the client staying connected and waiting for the entire duration of the work.

The client should be able to determine whether their request was received and is being processed, without needing to keep an open connection for the full length of that processing.

The system should preserve the outcome of the work (success, failure, or the completed report) so it can be retrieved after the fact, rather than only being deliverable at the exact moment generation finishes.

Report-generation duration should not determine how long the client is forced to wait before receiving any response at all.

A proposed revised contract

The current contract should be changed: the API should acknowledge accepted work before the report is complete, rather than returning 200 OK with the completed report in a single synchronous response.

Justification:

The current contract must be changed because it strictly couples the request lifetime to the work duration. As our data shows, the response time scaled almost exactly 1:1 with the artificial delay for the 0s, 3s, and 7s conditions. However, at the 12-second delay, the system failed entirely, returning an HTTP status of 000 and zero bytes because the work duration exceeded the server's timeout configuration. This proves that a synchronous design is unreliable for long-running tasks; it forces the client to hold an open connection, which will eventually be killed by network or server timeouts before the work is finished. Separating acknowledgement from completion would let the client know its request was received immediately, without report-generation time dictating how long it waits for any response, directly satisfying the derived requirements.